

PROJECT · HOLBIES FESTIVAL CONTROL

Holbies Festival Control

15 incidents. Une application qui tourne. Des contrats qui disent la vérité.

8	7	15
OBLIGATOIRES	BONUS	INCIDENTS

BRIEFING

Le festival ouvre bientôt. L'application existe déjà, mais plusieurs règles métier, contraintes de données et contrats d'API ne sont pas respectés. Votre mission n'est pas de tout reconstruire : vous devez enquêter, corriger la cause racine, prouver le comportement et éviter les régressions.

Le réflexe attendu

DOC → COMPORTEMENT RÉEL → ÉCART → DIAGNOSTIC → FIX → CHECK → INCIDENT LOG.

IMPORTANT

L'interface sert à observer les symptômes. Les visuels ne remplacent jamais les sources de vérité : règles métier, diagrammes Mermaid et contrat OpenAPI.

01 · BRIEFING

Vous êtes l'équipe de reprise

Holbies Festival accueille plusieurs scènes, des artistes, des bracelets, des zones à capacité limitée et une API utilisée par la régie. Le système est déjà en place. Il est également faux à quinze endroits.

CE QUI CHANGE

Cette fois, on ne vous demande pas « faites un MCD » ou « écrivez une API ». Les modèles corrects sont fournis. À vous de choisir lequel consulter pour comprendre un comportement incorrect.

Vos objectifs

- Lire une documentation comme une source de vérité.
- Diagnostiquer une cause racine sans confondre symptôme et bug.
- Réutiliser Merise, UML, REST/OpenAPI, JSON Schema, états-transitions et cohérence inter-modèles.
- Modifier du vrai code et utiliser le terminal pour valider.
- Documenter la correction dans un Incident Log.

RÈGLE

Un checker vert ne remplace pas l'explication. Chaque incident CLEARED doit avoir une cause et une correction compréhensibles dans `working/docs/INCIDENT_LOG.md`.

MODE CAMPAGNE

Le festival avance par actes

Vous arrivez dans la Control Room trois heures avant l'ouverture. Au lieu d'afficher quinze tickets d'un coup, Festival Control raconte la journée : chaque paire d'incidents stabilisée débloque le chapitre suivant.

ACTE 1 · 10:00

Soundcheck impossible

Régie programmation

FC-101 · FC-102

ACTE 2 · 11:05

Les portes s'entrouvrent

Sécurité / accréditations

FC-103 · FC-104

ACTE 3 · 12:10

La foule monte

Stage Manager

FC-105 · FC-106

ACTE 4 · 12:42

Le système ment

API / intégration

FC-107 · FC-108

ENCORE · 14:00+

After Hours

Control Room

FC-X01 → FC-X07

IMPORTANT

Le verrouillage des actes est uniquement narratif dans l'interface. Les checkers restent indépendants : si vous enquêtez autrement, la validation comportementale reste la même.

PROGRESSION & HISTORIQUE

Chaque commande laisse une trace

Les validations, les échecs de checker et les indices demandés alimentent automatiquement le Journal de régie. Un refresh ne remet donc pas l'histoire à zéro.

```
npm run check -- FC-101
```

Teste un incident et ouvre/ferme son statut.

```
npm run hint -- FC-101 1
```

Ajoute une demande d'indice au journal, sans pénalité.

```
npm run validate
```

Rejoue la validation globale et historise le résultat.

```
npm run reset
```

Réinitialise statuts + journal de campagne.

XP, rangs et badges

OBLIGATOIRE

100 XP par incident CLEARED.
8/8 débloquent GATES CLEARED
et le mode ENCORE.

BONUS

150 XP par incident bonus.
15/15 donne le badge PERFECT
RUN 15/15.

0 XP

RUNNER TECH

200 XP

RÉGIE JUNIOR

500 XP

OPS ENGINEER

800 XP

FESTIVAL ARCHITECT

1400 XP

ENCORE LEGEND

Le Journal de régie

10:13:22 ● **FAILED** FC-101 · checker échoué — le chevauchement est encore accepté.

10:18:41 ● **HINT** FC-101 · indice 1/3 demandé.

10:31:05 ● **CLEARED** FC-101 · Collision de scène — INCIDENT CLEARED (+100 XP).

12:55:12 ● **VALIDATION** Validation globale : 8/15 · 8/8 obligatoires.

RÈGLE DE JEU

Les XP sont un habillage ludique. Aucun point n'est retiré pour un indice et seul le checker comportemental valide réellement une correction.

02 · TERRITOIRE

Working pour vous. Private pour le moteur.

La séparation est volontaire. Vous devez pouvoir fouiller, modifier et casser votre espace sans toucher au système qui évalue votre travail.

```
Holbies_Festival_Control/  
├─ working/           # VOTRE ESPACE  
│  └─ app/            # application volontairement buggée  
│  └─ docs/           # règles + Mermaid + OpenAPI  
│  └─ tests/          # smoke tests visibles  
├─ private/          # NE PAS MODIFIER  
│  └─ checker.js      # validation comportementale  
│  └─ runtime/        # statut des incidents  
└─ package.json  
  └─ README.md
```

WORKING

Vous pouvez modifier tout ce qui est nécessaire sous `working/`. C'est votre atelier.

PRIVATE

Ne modifiez ni le checker, ni ses statuts, ni ses fichiers. Un incident n'est validé que par le comportement du système.

04 · DÉMARRAGE

Démarrer le projet sans se perdre

Tout se lance depuis la racine du dossier Holbies_Festival_Control, là où se trouve package.json. Le projet n'a aucune dépendance npm externe : Node.js 20+ suffit.

1 Ouvrir le projet

Décompressez le ZIP puis ouvrez un terminal dans le dossier Holbies_Festival_Control.

2 Vérifier Node.js

La commande `node -v` doit afficher une version 20 ou supérieure.

```
node -v
```

3 Lancer Festival Control

Gardez ce terminal ouvert pendant toute la session.

```
npm start
```

4 Ouvrir la plateforme

Dans le navigateur : `http://localhost:4177`

5 Ouvrir un second terminal

Utilisez-le pour les tests, checkers, indices et validations.

SECOND TERMINAL

`npm test` = smoke tests visibles. `npm run check -- FC-101` = validation d'un incident. `npm run validate` = validation globale.

À SAVOIR

Pas besoin de `npm install`. Si le port 4177 est occupé, vous pouvez lancer par exemple : `PORT=4180 npm start`.

05 · INTERFACE · TONIGHT

Le briefing vivant de la campagne

C'est l'écran d'arrivée. Il raconte l'acte courant, affiche les incidents actifs, la progression XP et le journal de régie.

Holbies
FESTIVAL 2026 · MATHIEU MOREL

0 / 8 obligatoires cleared

OUVERTURE
T-03:00

RUNNER TECH
0 XP / 1850

Tonight
Briefing — que faire maintenant

Le site
Line-up public, scènes, foule

Field Lab
Scanner les bracelets et piloter les scènes

Incidents
La liste des bugs à corriger

ACTE 1 / 4 · 10:00 · CAMPAGNE ACTIVE

SOUNDCHECK IMPOSSIBLE

• RÉGIE PROGRAMMATION

Le soundcheck démarre. Deux alertes apparaissent sur le planning : une scène est réservée deux fois et un artiste semble capable de jouer à deux endroits en même temps.

OBJECTIF DE L'ACTE
Stabiliser le line-up avant que les équipes techniques commencent les balances.

Voir le line-up

Tonight

Les bugs du line-up se voient sur **Le site**. Quand tu corriges, le site se met à jour tout seul.

FC-101 100 XP
Collision de scène
Main Stage : Neon Fox 18:00-19:00 et Syntax Error 18:30-19:30 se chevauchent.
• A voir sur le site / Field Lab
Voir le bug sur le site

FC-102 100 XP
Artiste ubiquitaire
Neon Fox apparaît sur Main Stage et Pulse Stage en même temps.
• A voir sur le site / Field Lab
Voir le bug sur le site

ACTE 1 / 4
SOUNDCHECK IMPOSSIBLE

ACTE 2 / 4
LES PORTES S'ENTROUVRENT

ACTE 3 / 4
LA FOULE MONTE

ACTE 4 / 4
LE SYSTÈME MENT

ENCORE
AFTER HOURS

JOURNAL
Aucune action encore. Lance ton premier checker.

À REPÉRER

À gauche : le contexte de l'acte. À droite : les incidents actuellement déverrouillés.

IMPORTANT

Les cartes changent avec les checkers : le front reflète la progression, mais seul le checker valide réellement une correction.

06 · INTERFACE · LE SITE

Voir les anomalies avant de toucher au code

Le site joue le rôle d'observatoire : line-up, états des scènes et occupation des zones montrent les symptômes visibles du système.

H

Holbies

FESTIVAL 2026 · MATHIEU MOREL

0 / 8 obligatoires cleared

OUVERTURE

T-03:00

RUNNER TECH

0 XP / 1850

Tonight

Briefing — que faire maintenant

Le site

Line-up public, scènes, foule

Field Lab

Scanner les bracelets et piloter les scènes

Incidents

La liste des bugs à corriger

Le site

Les anomalies ont un badge rouge. Quand tu corriges, les badges disparaissent.

VISIBLE MAINTENANT

Deux concerts se chevauchent sur la même scène

Un artiste est collé sur deux scènes en même temps

Un show CANCELLED est encore affiché au public

Club 42 est EVACUATED

Tonight

Field Lab

À VÉRIFIER

LINE-UP

Créneaux sensibles

18:00

SCHEDULED

Neon Fox

Main Stage · 19:00

CONFLIT 2 SCÈNES

18:30

SCHEDULED

Syntax Error

Main Stage · 19:30

CONFLIT

18:45

SCHEDULED

Neon Fox

Pulse Stage · 19:45

2 SCÈNES

20:00

SCHEDULED

Null Pointer

Club 42 · 21:00

21:00

CANCELLED

Les Holbies

Lab Stage · 22:00

ANNULÉ

STAGE OPS

État des scènes

READY

Main Stage

Capacité 12,000

READY

Pulse Stage

Capacité 8,000

EVACUATED

Club 42

Capacité 3,000

READY

Lab Stage

Capacité 2,000

ZONES

Occupation du site

General

STANDARD · VIP · CREW · ARTIST

14,120 / 25,000

VIP Deck

VIP · CREW · ARTIST

1,320 / 4,000

Backstage

CREW · ARTIST

211 / 600

Main Pit

STANDARD · VIP · CREW · ARTIST

4,999 / 5,000

À REPÉRER

Les badges rouges CONFLIT, 2 SCÈNES ou ANNULÉ signalent les anomalies visibles. Ils disparaissent quand la logique est corrigée.

RÉFLEXE

Observez d'abord le symptôme, puis confrontez-le aux règles et diagrammes fournis avant de modifier working/.

07 · INTERFACE · FIELD LAB

Un vrai laboratoire central

Le Field Lab n'est plus un simple simulateur d'accès. C'est le point de départ du diagnostic : cinq bancs d'essai permettent de reproduire un symptôme avec la vraie logique du projet avant d'ouvrir le code au hasard.

ACCÈS

SCÈNE

CAPACITÉ

PROGRAMMATION

PAYLOAD API

Holbies
FESTIVAL 2026 · MATHIEU MOREL

0 / 8 obligatoires cleared

OUVERTURE
T-03:00

RUNNER TECH
0 XP / 1850

Tonight
Briefing — que faire maintenant

Le site
Line-up public, scènes, foule

Field Lab
Reproduire les bugs avant de coder

Incidents
La liste des bugs à corriger

FIELD LAB

Cinq bancs d'essai. Zéro supposition.

Tu reproduis le symptôme avec la vraie logique, tu confrontes aux sources de vérité, puis tu corriges dans working/.

A TESTER MAINTENANT
Choisis un banc, reproduis, puis compare la réponse aux règles, diagrammes ou contrats.

General
14 120 / 25 000

VIP Deck
1 320 / 4 000

Backstage
211 / 600

Main Pit
4 999 / 5 000

BANC D'ESSAI

A Accès
Acte 2 : bracelets & zones

S Scène
Acte 3 : machine d'état

C Capacité
Acte 3 : valeurs limites

P Programmation
Acte 1 : créneaux & artistes

{ } Payload API
Acte 4 : contrat d'entrée

DRY RUN
Les essais n'altèrent pas la campagne.

BANC A · CONTROLE D'ACCES

Qui entre, et où ?

Choisis un bracelet et une zone. Le Lab exécute la vraie règle d'accès et affiche la décision brute.

1 BRACELET

WB-001 · STANDARD

2 ZONE

General (14120/25000)

3 Scanner le bracelet

Pas encore de scan.

En attente d'un scan.

CHECKER

npm run check -- PC-101 npm run hint -- PC-101 1 npm run validate private/hors zone

WORKFLOW

Reproduire → confronter aux règles / Mermaid / OpenAPI → corriger dans working/ → redémarrer npm start si la logique serveur a changé → rejouer → lancer le checker.

DRY RUN

Les cinq outils appellent la vraie logique métier sur une copie temporaire. Un test du Lab ne modifie ni les scènes, ni les zones, ni les shows, ni les bracelets de la campagne.

04 · VALIDATION

Comment un incident devient CLEARED

Le checker ne cherche pas une chaîne de code et ne vérifie pas si vous avez écrit le « bon if ». Il recrée ses propres données et teste le comportement attendu.

npm run check	3	npm run validate
COMMANDE	NIVEAUX INDICE	FINAL

Exemple

```
npm run check -- FC-101

FC-101 – Collision de scène
INCIDENT CLEARED
```

Un checker peut vérifier plusieurs propriétés en même temps : refus du cas invalide, maintien du cas valide et absence de mutation après un refus.

ANTI-CHEAT

Supprimer une ligne gênante dans seed . j son ne suffit pas : les checks reconstruisent leurs propres cas.

HOLBIES FESTIVAL CONTROL · APPLICATION

Le Field Lab

Cette zone sert à observer le système réel sans polluer la campagne. Choisissez un banc, reproduisez le comportement, puis comparez le résultat aux sources de vérité avant de corriger le code.

Vue du Field Lab central : le menu de gauche donne accès aux cinq bancs d'essai.

À REPÉRER

Accès = bracelets / zones · Scène = transitions FSM · Capacité = valeurs limites · Programmation = conflits de créneaux · Payload API = contrat JSON. Le badge DRY RUN confirme qu'aucun essai ne modifie l'état live.

05 · SOURCE DE VÉRITÉ

Les diagrammes sont corrects. Le code ne l'est pas.

Les fichiers de référence sont là pour vous aider à décider ce qui devrait se produire. Si le comportement réel contredit un diagramme ou une règle, c'est l'implémentation que vous devez questionner.

```
working/docs/  
├─ festival-rules.md  
├─ api-contract.yaml  
├─ incidents.md  
├─ INCIDENT_LOG.md  
└─ diagrams/  
    ├─ 01_data_model.mmd  
    ├─ 02_class_diagram.mmd  
    ├─ 03_access_sequence.mmd  
    ├─ 04_schedule_sequence.mmd  
    ├─ 05_stage_state.mmd  
    └─ 06_ticket_state.mmd
```

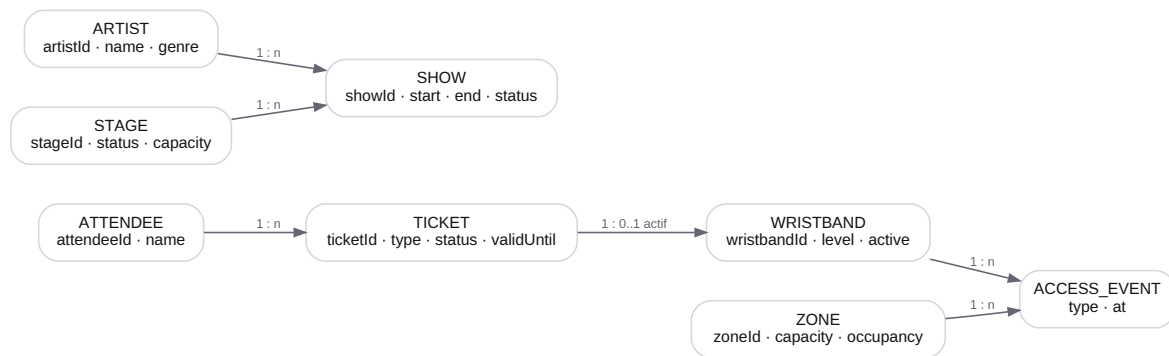
CONSEIL

Problème de relation ? Modèle de données. Problème de transition ? FSM. Payload ou code HTTP ? OpenAPI. Ordre des appels ? Séquence.

MERMAID · SOURCE DE VÉRITÉ

Modèle de données du festival

Les relations clés expliquent notamment pourquoi un bracelet dépend d'un ticket, pourquoi un show référence une scène et un artiste, et où se place l'historique des accès.



Représentation vectorielle du diagramme Mermaid · source : `working/docs/diagrams/01_data_model.mmd`

EXTRAIT MERMAID

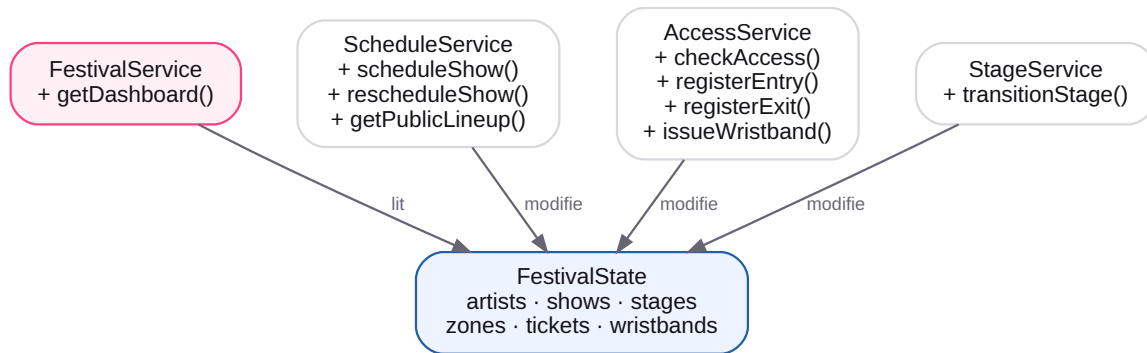
```

erDiagram
    ARTIST ||--o{ SHOW : "joue"
    STAGE ||--o{ SHOW : "accueille"
    TICKET ||--o| WRISTBAND : "active"
    WRISTBAND ||--o{ ACCESS_EVENT : "génère"
  
```

MERMAID · SOURCE DE VÉRITÉ

Architecture de services

L'application est volontairement petite : les services manipulent un état de festival commun. Les incidents ne nécessitent pas d'ajouter une nouvelle architecture complète.



Représentation vectorielle du diagramme Mermaid · source : working/docs/diagrams/02_class_diagram.mmd

EXTRAIT MERMAID

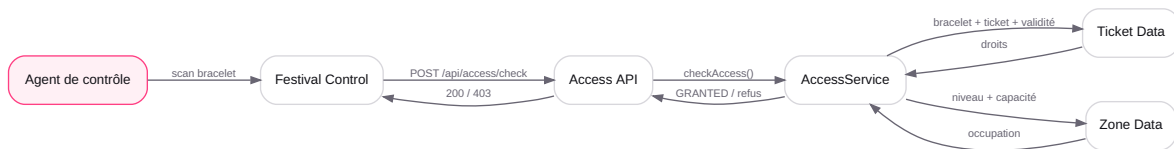
```

classDiagram
  class ScheduleService {
    +scheduleShow(show)
    +rescheduleShow(id, patch)
  }
  ScheduleService --> FestivalState
  
```

MERMAID · SOURCE DE VÉRITÉ

Séquence de contrôle d'accès

Un scan ne se résume pas au niveau du bracelet : l'identité du ticket, sa validité, les droits de zone et la capacité participent tous à la décision.



Représentation vectorielle du diagramme Mermaid · source : working/docs/diagrams/03_access_sequence.mmd

EXTRAIT MERMAID

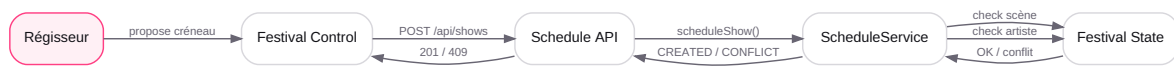
```

sequenceDiagram
    Agent->>UI: scanne un bracelet
    UI->>API: POST /api/access/check
    API->>Access: checkAccess(...)
    alt Accès autorisé
        API-->>UI: 200
    
```

MERMAID · SOURCE DE VÉRITÉ

Séquence de programmation

La création d'un show valide d'abord la disponibilité de la scène et de l'artiste. Le conflit est une erreur métier, pas une panne serveur.



Représentation vectorielle du diagramme Mermaid · source : `working/docs/diagrams/04_schedule_sequence.mmd`

EXTRAIT MERMAID

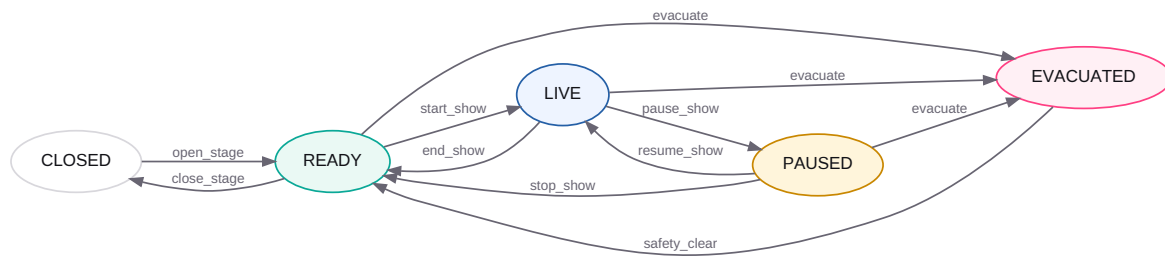
```

sequenceDiagram
    Ops->>UI: propose un nouveau créneau
    UI->>API: POST /api/shows
    API->>Service: scheduleShow(show)
    alt Aucun conflit
        API-->>UI: 201 Created
    end
  
```

MERMAID · SOURCE DE VÉRITÉ

Machine d'état d'une scène

La sécurité impose une transition explicite : une scène évacuée doit être déclarée sûre avant de pouvoir redevenir exploitable.



Représentation vectorielle du diagramme Mermaid · source : working/docs/diagrams/05_stage_state.mmd

EXTRAIT MERMAID

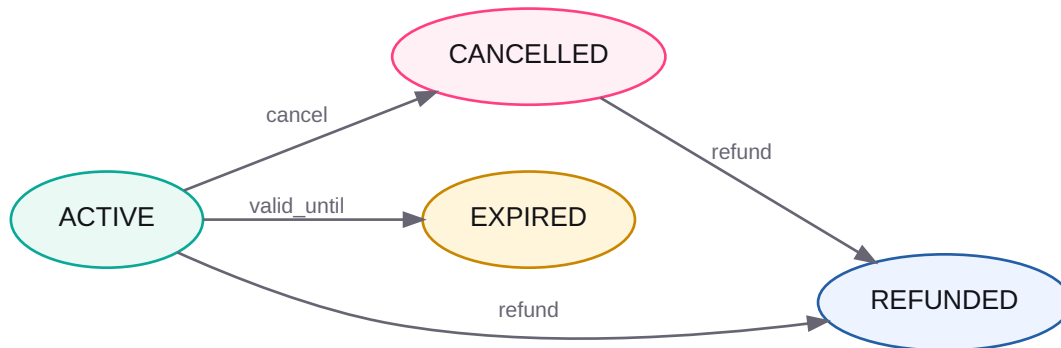
```

stateDiagram-v2
    CLOSED --> READY : open_stage
    READY --> LIVE : start_show
    LIVE --> EVACUATED : evacuate
    EVACUATED --> READY : safety_clear
  
```

MERMAID · SOURCE DE VÉRITÉ

Cycle de vie d'un billet

Le statut ACTIVE n'est qu'une partie du contrat : la date de validité intervient aussi lors du contrôle terrain.



Représentation vectorielle du diagramme Mermaid · source : working/docs/diagrams/06_ticket_state.mmd

EXTRAIT MERMAID

```
stateDiagram-v2
    ACTIVE --> CANCELLED : cancel
    ACTIVE --> EXPIRED : valid_until_reached
    ACTIVE --> REFUNDED : refund
```

12 · INCIDENT BOARD

Les incidents se déverrouillent avec l'histoire

Au départ, seuls FC-101 et FC-102 sont ouverts. Chaque paire d'incidents obligatoires stabilisée débloquent l'acte suivant. Les bonus restent verrouillés jusqu'à GATES CLEARED.

Holbies
 FESTIVAL 2026 · MATHIEU MOREL

0 / 8 obligatoires cleared

OUVERTURE
 T-03:00

RUNNER TECH
 0 XP / 1850

Tonight
 Briefing — que faire maintenant

Le site
 Line-up public, scènes, foule

Field Lab
 Scanner les bracelets et piloter les scènes

Incidents
 La liste des bugs à corriger

INCIDENT BOARD
Les anomalies de l'acte en cours
 Les actes futurs restent verrouillés. Les bonus s'ouvrent après les 8 obligatoires.

Tous

Obligatoires

Bonus

Tonight

FC-101 ACTE 1 · 100 XP Collision de scène Main Stage · Neon Fox 18:00-19:00 et Syntax Error 18:30-19:30 se chevauchent. · LINE-UP · Créneaux sensibles ● OPEN 	FC-102 ACTE 1 · 100 XP Artiste ubiquitaire Neon Fox apparaît sur Main Stage et Pulse Stage en même temps. · LINE-UP · Créneaux sensibles ● OPEN 	FC-103 ACTE 2 · 100 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-104 ACTE 2 · 100 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-105 ACTE 3 · 100 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED
FC-106 ACTE 3 · 100 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-107 ACTE 4 · 100 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-108 ACTE 4 · 100 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-X01 ENCORE · 150 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-X02 ENCORE · 150 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED
FC-X03 ENCORE · 150 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-X04 ENCORE · 150 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-X05 ENCORE · 150 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-X06 ENCORE · 150 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED 	FC-X07 ENCORE · 150 XP SIGNAL VERROUILLÉ Termine l'acte précédent pour ouvrir ce canal. ○ LOCKED

CAMPAGNE

8 incidents obligatoires ouvrent le festival. Les 7 bonus ENCORE permettent d'aller jusqu'au Perfect Run 15/15.

PAS DE RACCOURCI

Un incident verrouillé reste documenté dans le PDF, mais la progression narrative vous indique quoi traiter maintenant.

12 · INCIDENT BOARD

8 obligatoires + 7 ENCORE

Les incidents sont volontairement présentés comme des tickets terrain. On vous donne le symptôme et le critère de sortie, pas le fichier à modifier.

8	7	15
OBLIGATOIRES	BONUS	TOTAL

WORKFLOW

Pour chaque incident : reproduisez si utile dans l'app, consultez la référence pertinente, corrigez sous working/, lancez le checker, puis renseignez l'Incident Log.

Indices

Chaque incident possède trois niveaux d'indice progressifs :

```
npm run hint -- FC-101 1
npm run hint -- FC-101 2
npm run hint -- FC-101 3
```


INCIDENT OBLIGATOIRE · FC-101

Collision de scène

**À OBSERVER SUR
FESTIVAL
CONTROL**

Dans Festival Control, commencez par la zone LINE-UP - Créneaux sensibles. Sur Main Stage, Neon Fox est programmé de 18:00 à 19:00 et Syntax Error de 18:30 à 19:30. Pendant 30 minutes, la même scène est donc réservée à deux concerts différents.

Voici le problème

La régie ne doit jamais découvrir ce genre de collision quelques minutes avant l'ouverture. Le planning peut déjà contenir des données incohérentes, mais surtout le système ne doit pas permettre d'en créer de nouvelles. Votre objectif est de retrouver pourquoi la règle de disponibilité de la scène n'est pas protégée au moment où un show est ajouté.

Ce que le système doit garantir

Créer un nouveau show qui chevauche un show existant sur la même scène doit être refusé sans modifier le planning.

Source de vérité conseillée

Diagramme de séquence Schedule + règles métier.

Validation

```
npm run check -- FC-101
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

INCIDENT OBLIGATOIRE · FC-102

Artiste ubiquitaire

**À OBSERVER SUR
FESTIVAL
CONTROL**

Toujours dans le LINE-UP, suivez cette fois l'artiste Neon Fox. Il joue sur Main Stage de 18:00 à 19:00, puis apparaît aussi sur Pulse Stage à partir de 18:45. Les deux scènes sont différentes, mais l'artiste est le même.

Voici le problème

Le planning vérifie peut-être les ressources matérielles, mais il oublie une ressource essentielle : l'artiste lui-même. Un festival peut avoir plusieurs scènes en parallèle, pas cloner un groupe. Cherchez quel invariant métier manque lors de la programmation d'un show.

Ce que le système doit garantir

Un artiste ne peut pas être engagé sur deux shows simultanés.

Source de vérité conseillée

Modèle de données + règles métier.

Validation

```
npm run check -- FC-102
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

INCIDENT OBLIGATOIRE · FC-103

Bracelet fantôme

**À OBSERVER SUR
FESTIVAL
CONTROL**

Ouvrez le FIELD LAB - Contrôle d'accès. Sélectionnez le bracelet WB-002 - VIP, associé au billet de Bob, puis tentez un accès au VIP Deck. Festival Control accorde actuellement l'accès alors que le billet TK-002 est CANCELLED.

Voici le problème

Le bracelet semble être traité comme une autorisation autonome. Pourtant, il n'est qu'un support lié à un billet. Si le billet n'est plus valide, ses droits doivent disparaître immédiatement. Retrouvez à quel moment la chaîne bracelet -> ticket cesse d'être vérifiée.

Ce que le système doit garantir

Tout accès lié à un billet non ACTIVE doit être refusé.

Source de vérité conseillée

Séquence Access + machine d'état Ticket.

Validation

```
npm run check -- FC-103
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

INCIDENT OBLIGATOIRE · FC-104

Backstage breach

**À OBSERVER SUR
FESTIVAL
CONTROL**

Dans le FIELD LAB - Contrôle d'accès, prenez cette fois WB-003 - VIP, le bracelet valide de Chloé, et choisissez la zone Backstage. La plateforme laisse actuellement passer ce profil.

Voici le problème

VIP donne des avantages, mais pas tous les droits du festival. Le Backstage est réservé aux personnes qui travaillent ou se produisent sur l'événement. Le bug ne vient donc pas de la validité du billet : il vient de la manière dont le niveau d'accès est interprété.

Ce que le système doit garantir

VIP garde ses avantages mais ne franchit pas Backstage.

Source de vérité conseillée

Règles de zones + séquence Access.

Validation

```
npm run check -- FC-104
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

INCIDENT OBLIGATOIRE · FC-105

Scène morte

**À OBSERVER SUR
FESTIVAL
CONTROL**

Regardez d'abord STAGE OPS : Club 42 est en état EVACUATED. Puis ouvrez le FIELD LAB - Machine d'état, sélectionnez Club 42 et tentez directement le nouvel état LIVE. La plateforme accepte aujourd'hui cette transition.

Voici le problème

Une évacuation signifie qu'un incident de sécurité est en cours ou vient d'avoir lieu. Relancer un concert sans étape de remise en sécurité contourne totalement le processus opérationnel. La machine d'état fournie indique précisément le chemin autorisé : votre implémentation doit s'y conformer.

Ce que le système doit garantir

La transition doit respecter strictement la FSM fournie.

Source de vérité conseillée

Machine d'état Stage.

Validation

```
npm run check -- FC-105
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

INCIDENT OBLIGATOIRE · FC-106

Une personne de trop

À OBSERVER SUR FESTIVAL CONTROL

Dans le FIELD LAB, ouvrez le banc Capacité. Choisissez Main Pit puis testez limite - 1, limite exacte et limite + 1. Le Lab force temporairement l'occupation et simule l'entrée d'une personne sur une copie des données.

VOICI LE PROBLÈME

C'est un bug classique de valeur limite : le système sait refuser une zone déjà dépassée, mais peut encore accepter une entrée lorsque l'occupation est exactement égale à la capacité. Une seule comparaison incorrecte suffit à faire entrer la personne de trop.

CE QUE LE SYSTÈME DOIT GARANTIR

La dernière place disponible doit rester autorisée, mais dès que occupancy === capacity toute nouvelle entrée doit être refusée.

SOURCE DE VÉRITÉ

Règle de capacité + analyse des valeurs limites. Le Lab montre l'occupation avant et après sans modifier la zone live.

VALIDATION

```
npm run check -- FC-106
```

INCIDENT OBLIGATOIRE · FC-107

Payload sauvage

À OBSERVER SUR FESTIVAL CONTROL

Dans le FIELD LAB, ouvrez le banc Payload API. Testez d'abord le preset payload valide, puis niveau inconnu et champ surprise. Modifiez ensuite le JSON si nécessaire et comparez chaque réponse à `working/docs/api-contract.yaml`.

VOICI LE PROBLÈME

Le système accepte actuellement des données qui n'appartiennent pas au contrat public : niveaux d'autorisation inconnus ou propriétés supplémentaires. Le JSON peut sembler exploitable par le code tout en violant le contrat que les autres équipes utilisent.

CE QUE LE SYSTÈME DOIT GARANTIR

La création d'un bracelet doit être aussi stricte que le JSON Schema : propriétés attendues uniquement, identifiants au bon format et level limité aux valeurs prévues.

SOURCE DE VÉRITÉ

OpenAPI / JSON Schema : `working/docs/api-contract.yaml`. Le banc Payload API exécute `issueWristband()` sur un clone du festival.

VALIDATION

```
npm run check -- FC-107
```

INCIDENT OBLIGATOIRE · FC-108

Mauvais signal HTTP

À OBSERVER SUR FESTIVAL CONTROL

Dans le FIELD LAB, ouvrez le banc Programmation et reproduisez un conflit de créneau : le Lab montre la décision métier sur une copie des données. Ensuite, utilisez le terminal sur le vrai endpoint POST /api/shows pour comparer le code HTTP d'une création valide, d'un payload incomplet et d'un conflit au contrat OpenAPI.

VOICI LE PROBLÈME

Le corps JSON peut décrire correctement un échec alors que le code HTTP raconte autre chose. Une création, une erreur de forme et un conflit métier ne sont pas le même événement réseau. Un client doit pouvoir comprendre la catégorie du résultat sans interpréter le texte du JSON.

CE QUE LE SYSTÈME DOIT GARANTIR

Rétablir la sémantique REST : 201 pour une création, 400 pour une requête invalide et 409 pour un conflit métier, sans transformer ces cas en erreur serveur.

SOURCE DE VÉRITÉ

OpenAPI + sémantique HTTP. Le banc Programmation permet de reproduire visuellement un conflit avant le checker.

VALIDATION

```
npm run check -- FC-108
```


MODE ENCORE · FC-X01

Pass expiré

**À OBSERVER SUR
FESTIVAL
CONTROL**

Dans le FIELD LAB - Contrôle d'accès, testez le bracelet WB-004 - STANDARD de Diego vers la zone General. Son billet TK-004 porte encore le statut ACTIVE, mais sa date validUntil est déjà dépassée par rapport à la date du festival.

Voici le problème

Le statut administratif et la validité temporelle sont deux choses différentes. Un billet peut ne jamais avoir été annulé tout en étant devenu inutilisable. Le contrôle doit combiner ces deux informations avant d'accorder l'accès.

Ce que le système doit garantir

L'état et la validité temporelle doivent tous deux être vrais.

Source de vérité conseillée

Ticket state + séquence Access.

Validation

```
npm run check -- FC-X01
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

MODE ENCORE · FC-X02

Zone qui ne se vide jamais

À OBSERVER SUR FESTIVAL CONTROL

Observez les compteurs de zone dans le FIELD LAB, puis utilisez les endpoints d'entrée et de sortie depuis le terminal. Faites entrer un bracelet dans General, notez l'occupation, puis faites-le sortir. Le compteur final doit revenir à sa valeur de départ.

Voici le problème

Si les sorties ne libèrent pas réellement une place, Festival Control finit par considérer toutes les zones comme pleines alors que le public les a quittées. À l'inverse, une succession de sorties ne doit jamais produire une occupation négative. Les deux invariants doivent être vrais en même temps.

Ce que le système doit garantir

Une sortie doit libérer une place et ne jamais produire une occupation négative.

Source de vérité conseillée

Événements d'accès / invariants.

Validation

```
npm run check -- FC-X02
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

MODE ENCORE · FC-X03

Concert annulé toujours visible

**À OBSERVER SUR
FESTIVAL
CONTROL**

Revenez à la zone LINE-UP - Créneaux sensibles. Le show des Holbies sur Lab Stage est marqué CANCELLED, mais il est encore présenté comme un élément du programme public.

Voici le problème

L'annulation existe bien dans les données, mais la vue envoyée au public ne la respecte pas. Le problème n'est donc pas de supprimer l'historique du show : il faut distinguer les données internes de la projection réellement exposée aux festivaliers.

Ce que le système doit garantir

La vue publique ne doit exposer aucun show CANCELLED.

Source de vérité conseillée

Machine d'état Show implicite / vue.

Validation

```
npm run check -- FC-X03
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

MODE ENCORE · FC-X04

Reschedule non atomique

**À OBSERVER SUR
FESTIVAL
CONTROL**

Depuis le terminal, tentez de déplacer un show existant vers un créneau déjà occupé sur Main Stage. Le système détecte le conflit, mais vérifiez ensuite les données du show d'origine : elles ne doivent avoir changé ni de scène, ni d'heure.

Voici le problème

Un refus métier doit laisser le système dans l'état exact où il se trouvait avant la tentative. Si l'application modifie d'abord le show puis vérifie les conflits, elle peut retourner une erreur tout en ayant corrompu le planning. C'est précisément ce que l'atomicité doit empêcher.

Ce que le système doit garantir

Le reschedule doit valider les conflits et laisser le show intact en cas d'échec.

Source de vérité conseillée

Séquence Schedule + notion d'atomicité.

Validation

```
npm run check -- FC-X04
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

MODE ENCORE · FC-X05

Double bracelet

**À OBSERVER SUR
FESTIVAL
CONTROL**

Depuis le terminal, essayez d'émettre un second bracelet pour le billet TK-001, qui possède déjà WB-001 actif. Puis répétez le test après avoir désactivé l'ancien bracelet.

Voici le problème

Deux bracelets actifs pour un seul billet permettent de partager un droit d'entrée qui devrait être unique. Mais bloquer toute nouvelle émission serait également faux : un bracelet perdu doit pouvoir être remplacé après désactivation de l'ancien. La cardinalité métier doit donc porter sur les bracelets actifs.

Ce que le système doit garantir

Un seul bracelet actif par ticket ; un remplacement n'est possible qu'après désactivation.

Source de vérité conseillée

Cardinalités + règle d'unicité.

Validation

```
npm run check -- FC-X05
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

MODE ENCORE · FC-X06

Dérive sémantique API

**À OBSERVER SUR
FESTIVAL
CONTROL**

Interrogez `GET /api/artists/AR-001` depuis le terminal et comparez exactement les clés JSON obtenues avec le schéma `Artist` de `api-contract.yaml`. Le front attend `artistId`, `name` et `genre`.

Voici le problème

Le modèle interne peut employer ses propres conventions, mais l'API publique est un contrat stable. Renommer silencieusement `artistId` en `artist_id` ou `name` en `display_name` casse tous les consommateurs sans que la donnée elle-même soit fausse. Il faut restaurer le mapping public, pas réécrire tout le domaine.

Ce que le système doit garantir

Restaurer le contrat public exact sans renommer le modèle interne.

Source de vérité conseillée

OpenAPI + cohérence inter-modèles.

Validation

```
npm run check -- FC-X06
```

PREUVE

Le checker doit passer et votre `INCIDENT_LOG.md` doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

MODE ENCORE · FC-X07

Artiste orphelin

**À OBSERVER SUR
FESTIVAL
CONTROL**

Depuis le terminal, tentez de supprimer AR-001 - Neon Fox, puis inspectez le line-up et les données. Comparez ensuite avec la suppression de AR-999 - Unused Artist, qui n'est référencé par aucun show.

Voici le problème

Une suppression ne doit jamais casser l'intégrité du planning. Si un artiste est encore utilisé, son identifiant doit rester disponible pour les shows qui le référencent. En revanche, un artiste réellement inutilisé doit pouvoir être supprimé : le comportement attendu ressemble à une contrainte RESTRICT.

Ce que le système doit garantir

Appliquer un comportement de type RESTRICT sur un artiste encore utilisé.

Source de vérité conseillée

Intégrité référentielle / modèle de données.

Validation

```
npm run check -- FC-X07
```

PREUVE

Le checker doit passer et votre INCIDENT_LOG.md doit expliquer la cause racine et la correction appliquée.

ATTENTION

Ne corrigez pas uniquement la donnée de démonstration. Le checker reconstruit un cas indépendant et vérifie les non-régressions.

28 · INCIDENT LOG

Prouver que vous avez compris

Un incident n'est pas juste un voyant vert. Le journal doit permettre à un autre développeur de comprendre le problème sans relire toute votre diff.

```
## FC-101 – Collision de scène
```

```
Symptôme observé :
```

```
Cause racine :
```

```
Correction appliquée :
```

```
Pourquoi la correction respecte la documentation :
```

```
Commande de validation :
```

```
Résultat : CLEARED
```

PAS DE ROMAN

Quelques phrases précises suffisent. Évitez « j'ai corrigé le bug ». Nommez l'invariant qui n'était pas respecté.

29 · FINAL CHECK

Gates cleared ?

La validation finale rejoue les quinze incidents. Les huit obligatoires doivent tous être CLEARED pour autoriser l'ouverture. Les sept bonus permettent d'aller plus loin.

```
npm run validate
```

```
HOLBIES FESTIVAL – FINAL VALIDATION
```

```
FC-101 Collision de scène  
FC-102 Artiste ubiquitaire  
...  
FC-X07 Artiste orphelin
```

```
15/15 incidents cleared · 8/8 obligatoires  
GATES CLEARED – le festival peut ouvrir.
```

GO

Quand les 8 incidents obligatoires sont CLEARED, le festival peut ouvrir. Les bonus restent disponibles pour les équipes qui veulent pousser le diagnostic plus loin.

30 · MÉMO

Les commandes à garder sous la main

```
npm start          # lancer Festival Control
npm test           # smoke tests visibles
npm run check -- FC-101  # valider un incident
npm run hint -- FC-101 1  # indice 1 / 2 / 3
npm run validate    # rejouer les 15 incidents
npm run reset       # remettre le board en OPEN
```

Rappels

- Modifiez `working/`, pas `private/`.
- Les diagrammes Mermaid et `api-contract.yaml` sont corrects.
- Le système doit être robuste au cas général, pas uniquement aux données visibles.
- Un refus doit souvent laisser l'état exactement inchangé.
- Codes HTTP, cardinalités, capacités, états et noms publics font partie du contrat.

**DERNIER
RÉFLEXE**

Si vous ne savez pas où chercher, commencez par formuler l'invariant métier qui a été violé. Ensuite seulement choisissez le diagramme ou le contrat qui permet de le vérifier.